Arwyn Thandrayen [redacted]

1.Data Description and Pre-Processing

Our data set has numeric and categorical features with target feature 'price'. We drop the public reference column as this has no impact on the target. Then look at the statistical data, such as min, max values, mean, mode and median.

We can clean up the year_of_registration column. We know that the minimum year should be at least 1903 so any years before that are errors. We can use the reg_code column data to extrapolate the relevant year. For year of registration null values, where the vehicle condition is new and mileage is less than 200, we replace with the max year. For null values where the vehicle condition is used we can use the reg_code column to extrapolate the year. We can now drop the reg_code column.

```
[ ]  #Use reg_code column to substitute null value years(assuming reg code column is accurate)
     df.loc[(df['reg_code'] <= max_year-2000)
     & (df['year_of_registration'].isnull()), 'year_of_registration'] = 2000 + df['reg_code']

     df.loc[((df['reg_code'] > 50) & (df['reg_code'] < 50 + (max_year-2000)))
     & (df['year_of_registration'].isnull()), 'year_of_registration'] = 2001 - 50 + df['reg_code']
```

To deal with outliers, I cap the max mileage at roughly 140,000 using the Q3 + 1.75 IQR, since 99% of the dataset lies within this range. I also winsorize the price range while still including over 98% of the dataset. Standard scalers are sensitive to outliers so best to reduce them.

```
from scipy.stats.mstats import winsorize
df['price'] = winsorize(df['price'], limits=[0.015, 0.015])
df['price'].describe().round(2)
```

I change data type for crossover_car_and_van to integer for ease of working with models. We then take a sample of 10% of the dataset to train and test our models to save time. We split our data, with 80% to train and 20% to test.

```
# create sampe dataframe of 10% of oringial df
sample_df = df.sample(frac=0.1, random_state=42)  # 10% sample, random_state for reproducibility
sample_df.shape
```

We create a processor to process the data for our linear models. For numeric values, it will replace null values with the mean value and scale the data with a standard scaler. For categorical values, it replaces null values with the mode value and use target encoding to ensure string values are replaced with numerical ones.

```
# Fit and transform the data using the preprocessor
X_train_processed = preprocessor_lr.fit_transform(X_train, y_train)
X_test_processed = preprocessor_lr.transform(X_test)
```

2.Automated Feature Selection (10%): (All values are rounded to 3dp)

As a comparison we run a linear regression model without feature selection. We get a high test R-squared accuracy score of 0.808 and a score of 0.821 on the training data. Mean Absolute Error (MAE) test/train score of 3981.863 and 3847.809. (All outputs to 3dp). We want to establish which features have most impact on the target.

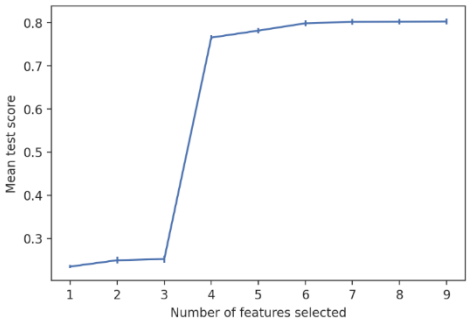
Using SelectBestK with K=6: We use this univariate feature selection which removes all but K highest scoring features. Here we try K=6. We create a pipeline with our preprocessor, SelectKBest and Linear Regression model, then fit the data. We can access the 6 best features selected by the model in order of importance according to their scores.

	Feature	Score
5	standard_model	51673.812
4	standard_make	18103.336
0	mileage	9892.208
1	year_of_registration	7639.557

The top 4 are Standard_model, standard_make, mileage and year_of_registration. With standard model scoring much higher indicating highest importance. Our model R^2 accuracy and MAE scores remain consistent with previous results showing no improvement.

With RFECV

Next we try Recursive feature elimination with cross-validation RFECV to select features. We construct a pipeline with the preprocessor and linear regressor and fit the data.



From the graph we can see that 4 features cause an increase in the model accuracy indicated by a slope. One particular feature has a very high impact. I was unable to ascertain which features due to difficulties with the ranking function.

The R^2 and MAE remain consistent with previous results.

Sequential Feature Selection (SFS) (Forward). This method adds features until they no longer improve model performance. This output shows the linear regressor works well with just 2 features, mileage and standard model.

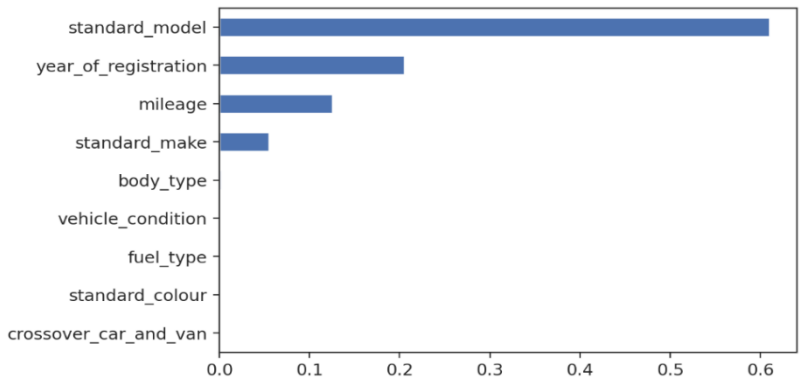
```
[92] sfs_forward.get_feature_names_out()

array(['mileage', 'standard_model'], dtype=object)
```

3.Tree Ensembles.

We adjust our preprocessor for a non-linear model which avoids scaling the numerical data and then run a **decision tree** with max depth 5 to set a base line performance. This outputs a test score of 0.831 and a training score of 0.845 (3dp). We create a **Random Forest** pipeline with max depth 5 and100 estimators. This averages the predictions of multiple independent decision trees and can address overfitting.

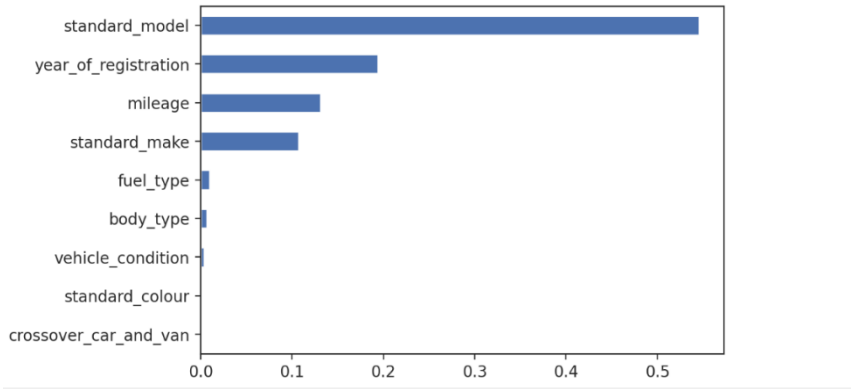
```
#Randomn Forest Tree Ensemble
rf_regr = RandomForestRegressor(
    n_estimators=100,
    max_depth=5, min_samples_split=20, min_samples_leaf=20,
    max_features=0.8
)
```

This outputs an improved test and training score of 0.872 and 0.880 respectively. Using feature importance and plotting the results, we see that standard model is by far the most important feature. This is consistent with our previous findings. However, there is some disagreement about the 2nd, 3rd and 4th most important feature.



Standard colour and crossover car and van again are shown to have little impact our target price. It is worth noting that standard model and standard make are not independent columns, some model names may be specific to and therefore an indicator of a particular make.

Boosted Trees works by building trees sequentially each one trying to correct the errors of the previous. We use similar parameters and get a noticeable improved R^2 test and training score, (0.914, 0.928) and a more nuanced result for feature importance that shows fuel type, body type and vehicle condition to also have some impact on the price.

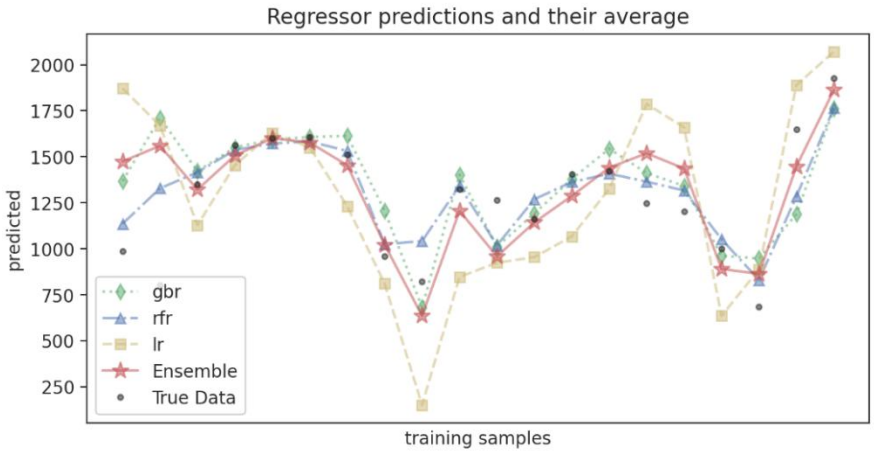


4. Model Ensembling.

Since we combine both linear and non-linear models in our pipeline, we use a preprocessor that scales the numeric features if the model is linear. We use gradient boosting (gbr), random forest (rfgr) and linear regressor (lr), to create an ensemble model using a **Voting Regressor**. This combines conceptually different regressors and returns the average predicted values. It is suitable for similarly performing models and should balance out individual weaknesses. We can see that the cross validated MAE has improved considerably, our ensemble model is doing a better job at predicting the target.

	test_mae_mean	test_mae_std	train_mae_mean	train_mae_std
gbr	3957.407997	142.605987	3574.387235	33.751335
rfgr	5274.410315	110.734764	4960.166415	64.119175
lr	6049.422875	129.514528	5799.056065	24.292969
ensemble	3037.753743	101.165193	2720.460597	16.196345

The graph shows how well the ensemble model fits the data.



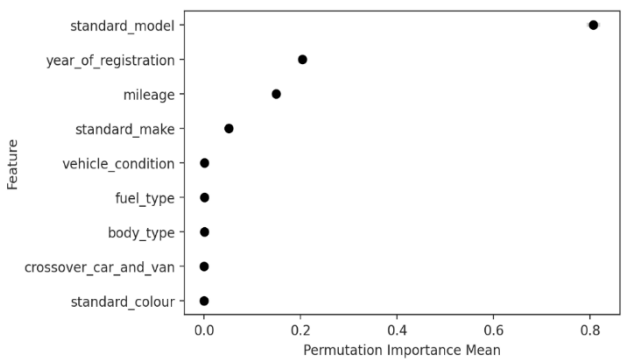
Stacking Regressor: This ensemble model stacks the predictions of each of our 3 regressor models and then combines them to one superior model to compute the final prediction. We see an R^2 test, train score of 0.911, 0.930 and more improvement in the MAE than with the voting regressor.

	test_mae_mean	test_mae_std	train_mae_mean	train_mae_std
gbr	4645.249489	232.537455	3672.893530	65.974391
rfgr	5615.878529	164.280094	4846.709646	62.322490
lr	6583.008502	205.861650	5924.923768	24.573897
ensemble	2726.048821	80.735522	2311.571271	39.411148

5. Feature Importance:

Permutation Importance: Permutation importance works by shuffling feature values in the test data and evaluating the impact on the model performance, in our case a random forest. Our results indicate highest importance to standard model followed by year of registration and mileage.

	feature	pi_mean	pi_std
3	standard_model	0.807	0.014
5	year_of_registration	0.204	0.003
0	mileage	0.149	0.007
2	standard_make	0.051	0.001
6	body_type	0.001	0.000



SHAP:

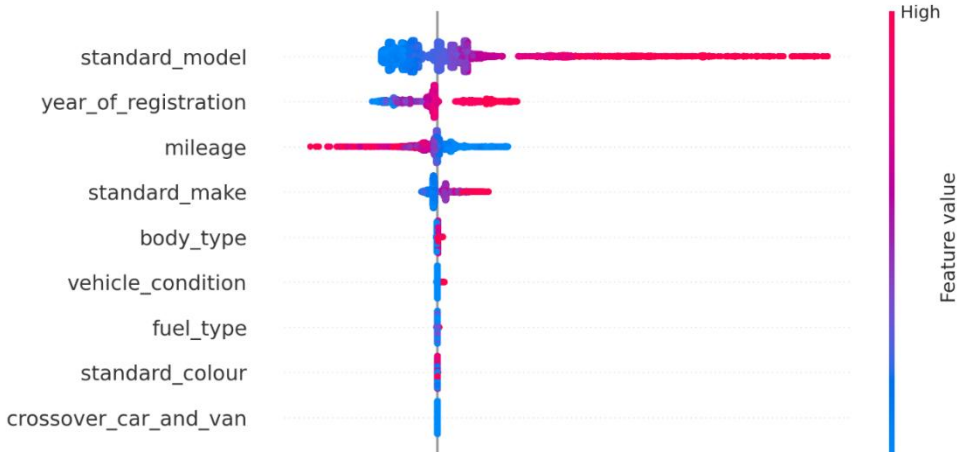
SHAP quantifies the impact of each feature on the target. Findings can easily be visualized to help understanding. Local explanations focus on explaining a single prediction rather than the entire model. This SHAP waterfall plot for a particular row [10], takes the average value for the dataset E[f(X)] = 15773.23 and then shows how each feature influences the target to arrive at the predicted value f(x)= 14681.61

The standard make has most impact in this instance. We find the corresponding row from the dataset for clarity on features.

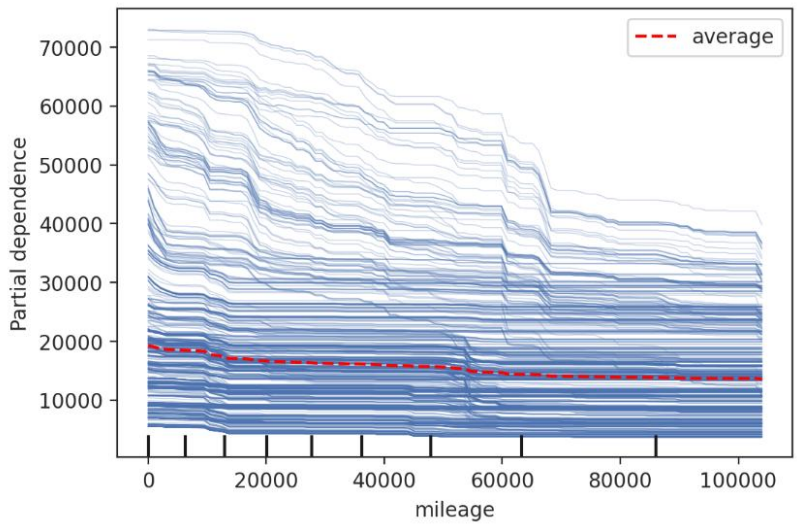
mileage	year_of_registration	crossover_car_and_van	standard_colour	standard_make	standard_model	vehicle_condition	body_type	fuel_type
51000.0	2015.0	0	White	Fiat	500X	USED	SUV	Diesel

6. SHAP/PDP Model Explanations: Useful tools to makes sense of complex data.

SHAP beeswarm plot gives a global or broader view of the entire data set showing standard model as the feature with highest importance. Each data point is an observation of the test data.



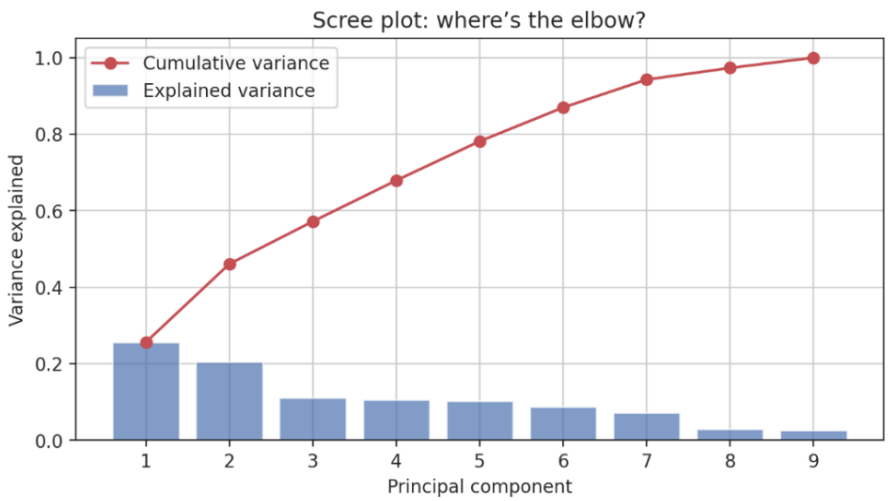
SHAP data can be easily manipulated.



Partial Dependence Plots (PDP). Individual Conditional Expectation (ICE) plots are where we take a particular row from the dataset and then vary just one feature such as mileage. For example take the used white Fiat 500X 2015 diesel SUV from our SHAP waterfall example and vary the mileage values and examine the impact on the target. The PDP represents the average of all the ICE's. We use the random forest regressor as our model. This graph shows that on average, but not always, mileage has a low impact on the target, and as it increases, it's impact on the target decreases.

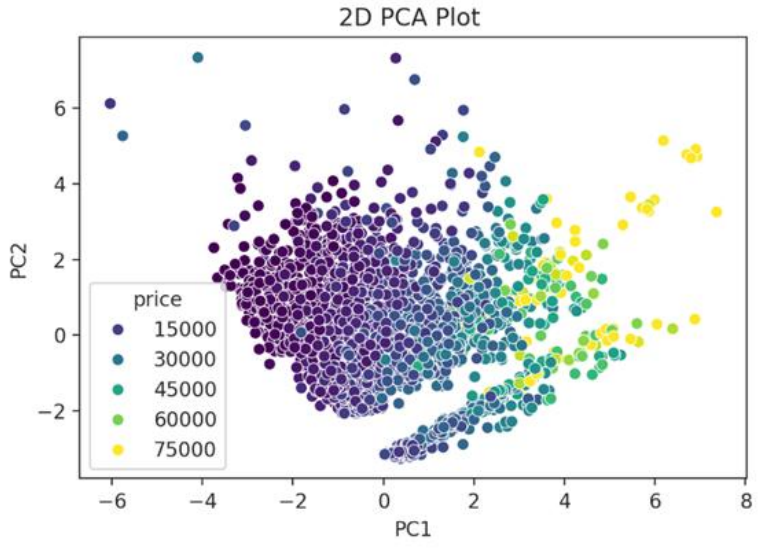
7.Dimensionality Reduction (Linear): Apply and analyse methods like PCA.

We take a smaller sample of the dataset for ease of visualisation methods. With PCA our aim is to simplify the dataset and reduce dimensions without losing important information. We apply PCA to our dataset to get the explained variance and then visualize with a scree plot. We find that nearly 60% of the variance can be explained by 3 features.



There is an elbow in the graph after features. This indciates we only need to keep 2 features.

Next, we project the data onto 2 components and for visualization**.**

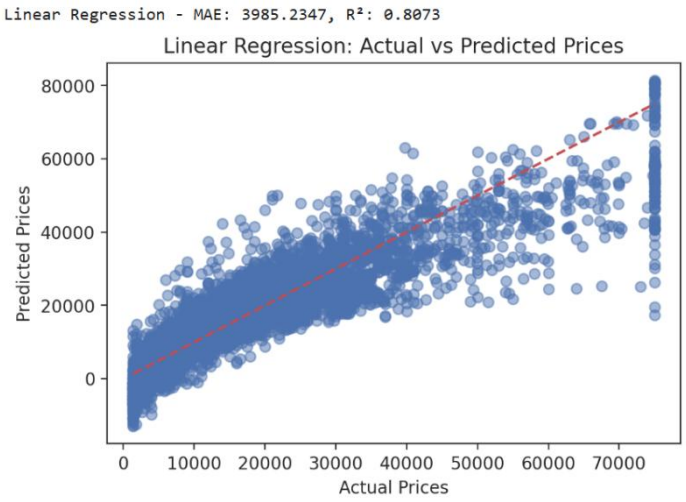


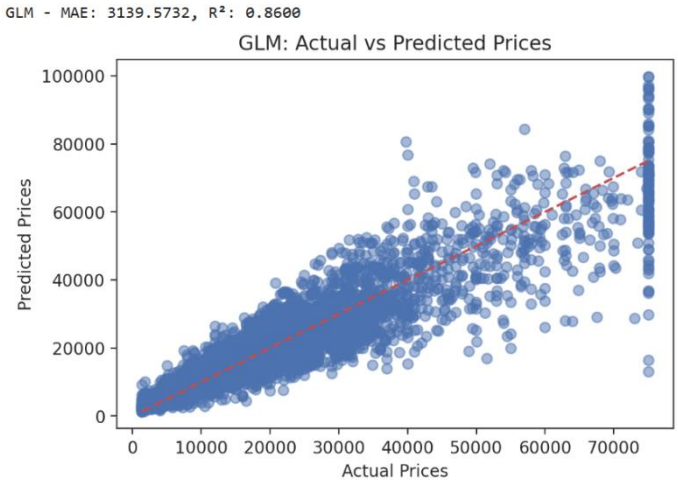
We see a trend of lower prices on the left (dark blue) with low PC1 values. As we move to the right the price increases shown by green and then yellow. There is a general separation of colours indicating that the features or characteristics contributing to the PC1 values correspond well to the different price ranges.

In essence, PC1 is a good predictor of price, summarizing a combination of variables from the original dataset that strongly influence the target.

9.**Polynomial Regression:**

We apply a basic linear regression model, creating a processor that scales both linear and encoded categorical data, and visualise the results. We then introduce nonlinearity by applying polynomial features and comparing the model results. We see very little difference visually but notice an improvement in the MAE and R^2 score. Thus adding polynomial features has improved the model by better capturing the relationship between features. I find that Mean Absolute Error (MAE) works better here than MSE mean squared error possibly due to the presence of outliers in our data.





A Generalized Linear Model (GLM) is able to capture non-linear behaviour by incorporating an exponential distribution for the Y variable, and a link function to the linear model. Our GLM is an improvement on the previous two models. A more robust alternative is Bayesian Regression. It incorporates confidence intervals and is less affected by outliers. However, it may not be suitable for this dataset as on examination the performance was not as good as the other models.

Model	**MAE** (lower is best)	**R^2** (higher is best)
Linear basic	3985.235	0.807
Linear(poly)	3603.602	0.837
GLM	3139.573	0.860
Bayesian	4569.22	0.74